

LAB 2.1: Write a Python program using the Pandas library to perform data cleaning on an employee dataset.

Theory:

Data cleaning is the process of preparing raw data by removing missing values, duplicates, and correcting inconsistent data. It ensures data quality before analysis. The Pandas library provides functions like `dropna()`, `drop_duplicates()`, and `replace()` to clean datasets efficiently.

Code:

```
import pandas as pd

# Read CSV file
employee_data = pd.read_csv('/content/employee1.csv')
print("\nOriginal Employee Data\n")
print(employee_data.head(10))

# Remove rows with missing values
employee_data = employee_data.dropna()

# Remove duplicate rows
employee_data = employee_data.drop_duplicates()

# Delete Bonus % column
employee_data = employee_data.drop('Bonus %', axis=1)

# Fix inconsistent team names
employee_data['Team'] = employee_data['Team'].replace({
    'Fin': 'Finance',
    'Mkt': 'Marketing',
    'Financeance': 'Finance'
})

print("\nCleaned Employee Data\n")
print(employee_data.head(10))

# Save cleaned data into a new CSV file
employee_data.to_csv('employees_cleaned.csv', index=False)
print("\nCSV file saved successfully!\n")
print("Name: Alisha Khatri ")
print("Roll no: 03 \n")
```

Output:

...

Original Employee Data

	FirstName	Gender	Salary	Bonus %	Team
0	Jerry	Male	138705	9.340	Fin
1	Louise	Female	63241	15.132	NaN
2	Douglas	Male	97308	6.945	Marketing
3	Douglas	Male	97308	6.945	Marketing
4	Matthew	Male	100612	13.645	Mkt
5	Scott	NaN	122367	5.218	Legal

Cleaned Employee Data

	FirstName	Gender	Salary	Team
0	Jerry	Male	138705	Finance
2	Douglas	Male	97308	Marketing
4	Matthew	Male	100612	Marketing

CSV file saved successfully!

Name: Alisha Khatri
Roll no: 03

LAB 2.2: Write a Python program using the Pandas library to handle missing values in an employee dataset.

Theory:

Missing values reduce data accuracy. They can be handled using:

- Mean/median substitution
 - Linear interpolation
- In Pandas, `fillna()` replaces missing values with statistical measures, while `interpolate()` estimates values based on surrounding data points.

Code:

```
import pandas as pd

# Read employee dataset
employee_data = pd.read_csv('employees.csv')
print("Original Employee Data")
print(employee_data.head(25))

# Fill missing salary values using mean
employee_data['Salary'] = employee_data['Salary'].fillna(
employee_data['Salary'].mean()
)
print("\nData After Filling Missing Salary with Mean")
print(employee_data.head(25))

# Reload dataset for interpolation method
employee_data2 = pd.read_csv('employees.csv')
print("\nOriginal Data for Interpolation Method")
print(employee_data2.head(25))

# Fill missing salary values using linear interpolation
employee_data2['Salary'] = employee_data2['Salary'].interpolate(method='linear')
print("\nData After Linear Interpolation")
print(employee_data2.head(25))

# Save cleaned datasets
employee_data.to_csv('employees_mean_cleaned.csv', index=False)
employee_data2.to_csv('employees_interpolated.csv', index=False)
print("\nFiles Saved Successfully\n")
print("Name: Alisha Khatri ")
print("Roll no: 03 \n")
```

Output:

```

Original Employee Data
...
0 Douglas Male 97308.0 6.945 Marketing
1 Douglas Male 97308.0 6.945 Marketing
2 Douglas Male 97310.0 6.945 Marketing
3 Thomas Male 61933.0 4.170 NaN
4 Maria Female 130590.0 11.858 Finance
5 Jerry Male 138705.0 9.340 Fin
6 Larry Male NaN 1.389 Client Services
7 Dennis Male 115163.0 10.125 Legal
8 Ruby Female 65476.0 10.012 Product
9 NaN Female 45906.0 11.598 Fin
10 Angela Female 95570.0 18.523 Engineering
11 Maria Female 130590.0 11.858 Finance
12 Frances Female 139852.0 7.524 Business Development
13 Louise Female 63241.0 15.132 NaN
14 Julie Female 102508.0 12.637 Legal
15 Brandon Male 112807.0 17.492 Human Resources
16 Gary Male 109831.0 5.831 Sales
17 Kimberly Female NaN 14.543 Finance
18 Lillian Female 59414.0 1.256 Product
19 Jeremy Male 90370.0 7.369 Human Resources
20 Shawn Male 111737.0 6.414 Product
21 Diana Female 132940.0 19.082 Client Services
22 Donna Female 81014.0 1.894 Product
23 Lois NaN 64714.0 4.934 Legal
24 Matthew Male 100612.0 13.645 Mkt

Data After Filling Missing Salary with Mean
0 Douglas Male 97308.000000 6.945 Marketing
1 Douglas Male 97308.000000 6.945 Marketing
2 Douglas Male 97310.000000 6.945 Marketing
3 Thomas Male 61933.000000 4.170 NaN
4 Maria Female 130590.000000 11.858 Finance
5 Jerry Male 138705.000000 9.340 Fin
6 Larry Male 90754.204795 1.389 Client Services
7 Dennis Male 115163.000000 10.125 Legal
8 Ruby Female 65476.000000 10.012 Product
9 NaN Female 45906.000000 11.598 Fin
10 Angela Female 95570.000000 18.523 Engineering
11 Maria Female 130590.000000 11.858 Finance
12 Frances Female 139852.000000 7.524 Business Development
13 Louise Female 63241.000000 15.132 NaN
14 Julie Female 102508.000000 12.637 Legal
15 Brandon Male 112807.000000 17.492 Human Resources
16 Gary Male 109831.000000 5.831 Sales
17 Kimberly Female 90754.204795 14.543 Finance
18 Lillian Female 59414.000000 1.256 Product
19 Jeremy Male 90370.000000 7.369 Human Resources
20 Shawn Male 111737.000000 6.414 Product
21 Diana Female 132940.000000 19.082 Client Services
22 Donna Female 81014.000000 1.894 Product
23 Lois NaN 64714.000000 4.934 Legal
24 Matthew Male 100612.000000 13.645 Mkt

```

```
... Original Data for Interpolation Method
First Name Gender Salary Bonus % Team
0 Douglas Male 97308.0 6.945 Marketing
1 Douglas Male 97308.0 6.945 Marketing
2 Douglas Male 97310.0 6.945 Marketing
3 Thomas Male 61933.0 4.170 NaN
4 Maria Female 130590.0 11.858 Finance
5 Jerry Male 138705.0 9.340 Fin
6 Larry Male NaN 1.389 Client Services
7 Dennis Male 115163.0 10.125 Legal
8 Ruby Female 65476.0 10.012 Product
9 NaN Female 45906.0 11.598 Fin
10 Angela Female 95570.0 18.523 Engineering
11 Maria Female 130590.0 11.858 Finance
12 Frances Female 139852.0 7.524 Business Development
13 Louise Female 63241.0 15.132 NaN
14 Julie Female 102508.0 12.637 Legal
15 Brandon Male 112807.0 17.492 Human Resources
16 Gary Male 109831.0 5.831 Sales
17 Kimberly Female NaN 14.543 Finance
18 Lillian Female 59414.0 1.256 Product
19 Jeremy Male 90370.0 7.369 Human Resources
20 Shawn Male 111737.0 6.414 Product
21 Diana Female 132940.0 19.082 Client Services
22 Donna Female 81014.0 1.894 Product
23 Lois NaN 64714.0 4.934 Legal
24 Matthew Male 100612.0 13.645 Mkt
```

```
Data After Linear Interpolation
First Name Gender Salary Bonus % Team
0 Douglas Male 97308.0 6.945 Marketing
1 Douglas Male 97308.0 6.945 Marketing
2 Douglas Male 97310.0 6.945 Marketing
3 Thomas Male 61933.0 4.170 NaN
4 Maria Female 130590.0 11.858 Finance
5 Jerry Male 138705.0 9.340 Fin
6 Larry Male 126934.0 1.389 Client Services
7 Dennis Male 115163.0 10.125 Legal
8 Ruby Female 65476.0 10.012 Product
9 NaN Female 45906.0 11.598 Fin
10 Angela Female 95570.0 18.523 Engineering
11 Maria Female 130590.0 11.858 Finance
12 Frances Female 139852.0 7.524 Business Development
13 Louise Female 63241.0 15.132 NaN
14 Julie Female 102508.0 12.637 Legal
15 Brandon Male 112807.0 17.492 Human Resources
16 Gary Male 109831.0 5.831 Sales
17 Kimberly Female 84622.5 14.543 Finance
18 Lillian Female 59414.0 1.256 Product
19 Jeremy Male 90370.0 7.369 Human Resources
20 Shawn Male 111737.0 6.414 Product
21 Diana Female 132940.0 19.082 Client Services
22 Donna Female 81014.0 1.894 Product
23 Lois NaN 64714.0 4.934 Legal
24 Matthew Male 100612.0 13.645 Mkt
```

Files Saved Successfully

LAB 2.3: Write a Python program using NetworkX and Matplotlib to create and analyze a signed social network.

Theory:

A signed network is a graph where edges have positive (+) or negative (-) relationships. It is used to analyze social balance theory. Stable triangles have either all positive edges or one positive edge, while others are unstable. NetworkX is used to model graphs and study relationships.

Code:

```
import networkx as nx

import matplotlib.pyplot as plt
import random
import itertools

# Function to get signs of all triangles
def get_triangle_signs(graph, triangle_list):
    signs_data = []
    for triangle in triangle_list:
        temp = []
        temp.append(graph[triangle[0]][triangle[1]]['sign'])
        temp.append(graph[triangle[1]][triangle[2]]['sign'])
        temp.append(graph[triangle[2]][triangle[0]]['sign'])
        signs_data.append(temp)
    return signs_data

# Function to count unstable triangles
def count_unstable_triangles(signs_data):
    stable = 0
    for sign in signs_data:
        plus_count = sign.count('+')
        if plus_count == 1 or plus_count == 3:
            stable += 1
    unstable = len(signs_data) - stable
    return unstable

# Function to make graph stable
def stabilize_graph(graph, triangle_list, signs_data):
    unstable_found = False
    while not unstable_found:
        random_triangle = random.randint(0, len(triangle_list) - 1)
```

```

if signs_data[random_triangle].count('+') % 2 == 0:
    unstable_found = True
    edge_choice = random.randint(1, 3)
    triangle = triangle_list[random_triangle]
    # Change sign of one edge
    if edge_choice == 1:
        current = graph[triangle[0]][triangle[1]]['sign']
        graph[triangle[0]][triangle[1]]['sign'] = '-' if current == '+' else '+'
    elif edge_choice == 2:
        current = graph[triangle[1]][triangle[2]]['sign']
        graph[triangle[1]][triangle[2]]['sign'] = '-' if current == '+' else '+'
    else:
        current = graph[triangle[0]][triangle[2]]['sign']
        graph[triangle[0]][triangle[2]]['sign'] = '-' if current == '+' else '+'
    return graph
# Function to form coalitions
def create_coalitions(graph):
    group1 = []
    group2 = []
    start_node = random.choice(list(graph.nodes()))
    group1.append(start_node)
    visited = []
    queue = [start_node]
    for node in queue:
        if node not in visited:
            neighbors = list(graph.neighbors(node))
            for neighbor in neighbors:
                if graph[node][neighbor]['sign'] == '+':
                    if neighbor not in group1:
                        group1.append(neighbor)
                    if neighbor not in queue:
                        queue.append(neighbor)
            else:
                if neighbor not in group2:
                    group2.append(neighbor)
            visited.append(node)
    return group1, group2

```

```

# Create graph
graph = nx.Graph()
n = 8
graph.add_nodes_from(range(1, n + 1))
node_names = {
    1: "A", 2: "B", 3: "C", 4: "D",
    5: "E", 6: "F", 7: "G", 8: "H"
}
graph = nx.relabel_nodes(graph, node_names)
# Add edges with random signs
signs = ['+', '-']
for i in graph.nodes():
    for j in graph.nodes():
        if i != j:
            graph.add_edge(i, j, sign=random.choice(signs))
# Display original graph
edge_labels = nx.get_edge_attributes(graph, 'sign')
position = nx.circular_layout(graph)
nx.draw(graph, position, with_labels=True, node_size=3000)
nx.draw_networkx_edge_labels(
    graph,
    position,
    edge_labels=edge_labels,
    font_color='blue'
)
plt.title("Original Signed Network")
plt.show()
# Find all triangles
triangle_list = [list(x) for x in itertools.combinations(graph.nodes(), 3)]
# Get triangle signs
triangle_signs = get_triangle_signs(graph, triangle_list)
# Count unstable triangles
unstable_count = count_unstable_triangles(triangle_signs)
print("Initial Unstable Triangles:", unstable_count)
# Make graph stable
while unstable_count != 0:
    graph = stabilize_graph(graph, triangle_list, triangle_signs)

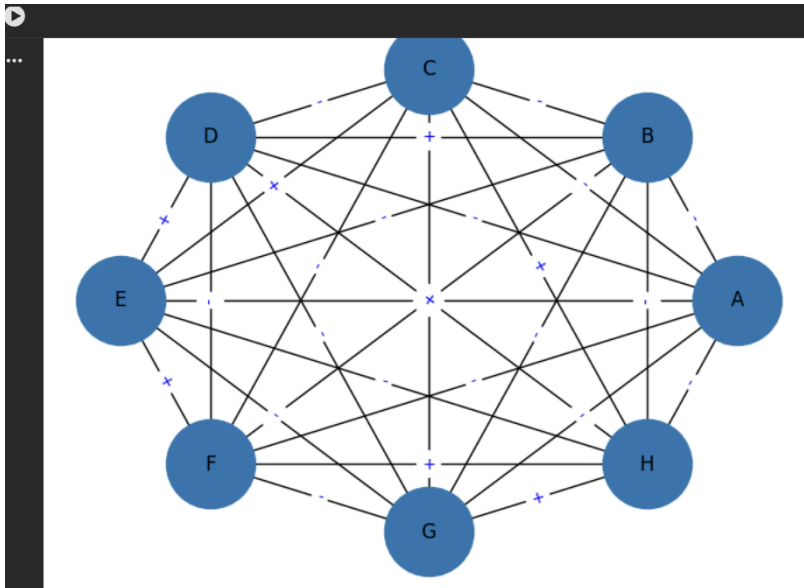
```

```

triangle_signs = get_triangle_signs(graph, triangle_list)
unstable_count = count_unstable_triangles(triangle_signs)
# Create coalitions
coalition1, coalition2 = create_coalitions(graph)
print("Coalition 1:", coalition1)
print("Coalition 2:", coalition2)
# Draw final stable graph
edge_labels = nx.get_edge_attributes(graph, 'sign')
nx.draw_networkx_nodes(
graph,
position,
nodelist=coalition1,
node_color='red',
node_size=3500
)
nx.draw_networkx_nodes(
graph,
position,
nodelist=coalition2,
node_color='lightblue',
node_size=3500
)
nx.draw_networkx_edges(graph, position)
nx.draw_networkx_labels(graph, position)
nx.draw_networkx_edge_labels(
graph,
position,
edge_labels=edge_labels,
font_color='red'
)
plt.title("Stable Signed Network with Coalitions")
plt.show()
print("Name: Alisha Khatri ")
print("Roll no: 03 \n")

```

Output:

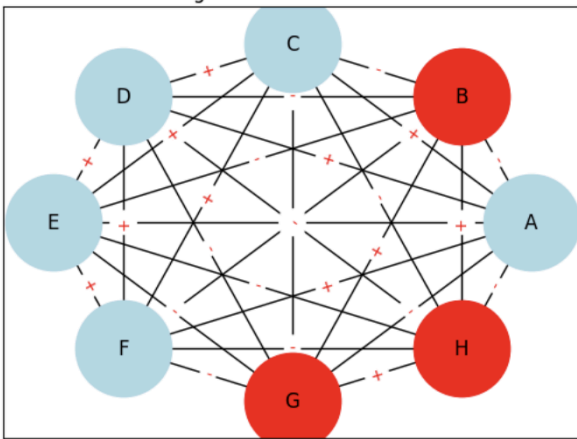


Initial Unstable Triangles: 36

Coalition 1: ['B', 'G', 'H']

Coalition 2: ['A', 'C', 'D', 'E', 'F']

Stable Signed Network with Coalitions



Name: Alisha Khatri
Roll no: 03

LAB 2.4: Write a Python program using the Pandas library to create a new variable in a student dataset.

Theory:

This process creates a new feature (feature engineering) by calculating age from DOB using current date. It helps in better data analysis and prediction. Pandas datetime functions and Python's `datetime` module are used for conversion and calculation.

Code:

```
import pandas as pd

from datetime import datetime
from dateutil.relativedelta import relativedelta

# Read student dataset
student_data = pd.read_csv('student-data.csv')
# Function to calculate age from date of birth
def calculate_age(dob):
    current_date = datetime.now()
    age = relativedelta(current_date, dob).years
    return age

# Convert dob column into datetime format
student_data['dob'] = pd.to_datetime(student_data['dob'])
# Create new age column
student_data['Age'] = student_data['dob'].apply(calculate_age)
# Select required columns
final_data = student_data[
['name', 'dob', 'Age', 'is_student', 'target']
]
print("Student Data with Age")
print(final_data)
# Save updated dataset
final_data.to_csv('student_age_data.csv', index=False)
print("\nNew CSV File Saved Successfully")
print("Name: Alisha Khatri ")
print("Roll no: 03 \n")
```

Output:

```
... Student Data with Age
      name      dob Age is_student target
0   Thomas Crosby 1996-07-16   29     False  809.97
1   Brandon Reeves 1957-11-30   68     False  866.41
2   Maria Castillo 1981-02-03   45     False  776.92
3     Carol Jones 1990-09-17   35      True  929.72
4  Robert Thompson 1967-06-09   58      True  923.66
..      ...      ...
99   Barbara Ford 1975-01-23   51     False  937.97
100  Maria Castillo 1981-02-03   45     False  776.92
101   Edward Hood 1984-01-10   42     False  975.77
102   Jeffrey Smith 1996-09-29   29     False  996.08
103  Hayley Hoffman 1958-06-23   67     False  825.04
```

```
[104 rows x 5 columns]
```

```
New CSV File Saved Successfully
```

```
Name: Alisha Khatri
```

```
Roll no: 03
```

LAB 2.5: Write a Python program using the Apriori algorithm to perform association rule mining on a store dataset.

Theory:

Apriori is a data mining algorithm used to find frequent itemsets and generate rules. It uses support, confidence, and lift to measure rule strength. It is commonly used in market basket analysis. Apriori helps discover relationships between items.

Code:

```
import pandas as pd
from apyori import apriori

# Read dataset
store_data = pd.read_csv('store_data.csv', header=None)
print("Store Dataset")
print(store_data.head())
# Convert dataset into list format
transactions = []
for i in range(0, 7501):
    transaction = []
    for j in range(0, 10):
        transaction.append(str(store_data.values[i, j]))
    transactions.append(transaction)
# Apply Apriori Algorithm
rules = apriori(

transactions,
min_support=0.0045,
min_confidence=0.2,
min_lift=3,
min_length=2
)
# Convert rules into list
results = list(rules)
print("\nAssociation Rules\n")
# Display rules
for item in results:
    pair = item[0]
```

```
items = list(pair)
print("Rule:", items[0], "->", items[1])
print("Support:", item[1])
print("Confidence:", item[2][0][2])
print("Lift:", item[2][0][3])
print("-----")
print("Name: Alisha Khatri ")
print("Roll no: 03 \n")
```

Output:

Store Dataset

```

0      0      1      2      3      4  \
0      shrimp almonds avocado vegetables mix green grapes
1      burgers meatballs eggs NaN NaN
2      chutney NaN NaN NaN NaN
3      turkey avocado NaN NaN NaN
4 mineral water milk energy bar whole wheat rice green tea

```

```

5      6      7      8      9  \
0 whole weat flour yams cottage cheese energy drink tomato juice
1 NaN NaN NaN NaN NaN NaN
2 NaN NaN NaN NaN NaN NaN
3 NaN NaN NaN NaN NaN NaN
4 NaN NaN NaN NaN NaN NaN

```

```

10      11      12      13      14      15  \
0 low fat yogurt green tea honey salad mineral water salmon
1 NaN NaN NaN NaN NaN NaN
2 NaN NaN NaN NaN NaN NaN
3 NaN NaN NaN NaN NaN NaN
4 NaN NaN NaN NaN NaN NaN

```

```

16      17      18      19
0 antioxydant juice frozen smoothie spinach olive oil
1 NaN NaN NaN NaN NaN
2 NaN NaN NaN NaN NaN
3 NaN NaN NaN NaN NaN
4 NaN NaN NaN NaN NaN

```

Association Rules

Rule: mushroom cream sauce -> escalope

Support: 0.005465937874950006

Confidence: 0.2992700729927007

Lift: 3.8770722236930015

Rule: pasta -> escalope

Support: 0.005865884548726837

Confidence: 0.3728813559322034

Lift: 4.830713386610462

Rule: ground beef -> herb & pepper

Support: 0.015997866951073192

Confidence: 0.3234501347708895

Lift: 3.2919938411349285

Rule: tomato sauce -> ground beef

Support: 0.005332622317024397

Confidence: 0.3773584905660377

Lift: 3.840659481324083

Rule: olive oil -> whole wheat pasta

Support: 0.007998933475536596

Confidence: 0.2714932126696833

Lift: 4.147597939379418

Rule: shrimp -> pasta

Support: 0.005065991201173177

Confidence: 0.3220338983050847

Rule: shrimp -> frozen vegetables
Support: 0.004932675643247567
Confidence: 0.224242424242427
Lift: 3.1381388511985526

Rule: spaghetti -> cooking oil
Support: 0.004532728969470737
Confidence: 0.5573770491803278
Lift: 3.201290387367258

Rule: mushroom cream sauce -> nan
Support: 0.005199306759098787
Confidence: 0.28467153284671537
Lift: 3.92522735079434

Rule: pasta -> nan
Support: 0.005865884548726837
Confidence: 0.3728813559322034
Lift: 5.141512961116651

Rule: spaghetti -> frozen vegetables
Support: 0.008665511265164644
Confidence: 0.31100478468899523
Lift: 3.165328208890303

Rule: frozen vegetables -> olive oil
Support: 0.004666044527396347
Confidence: 0.42168674698795183
Lift: 3.2541895978977644

Rule: shrimp -> frozen vegetables
Support: 0.007199040127982935
Confidence: 0.3068181818181818
Lift: 3.218801652892562

Rule: shrimp -> frozen vegetables
Support: 0.005999200106652446
Confidence: 0.21531100478468898
Lift: 3.0131489680782684

Rule: spaghetti -> frozen vegetables
Support: 0.006665777896280496
Confidence: 0.23923444976076558
Lift: 3.4980460188216425

Rule: spaghetti -> grated cheese
Support: 0.005332622317024397
Confidence: 0.3225806451612903
Lift: 3.283144395325426

Rule: mineral water -> ground beef
Support: 0.006665777896280496
Confidence: 0.39062500000000006
Lift: 3.975682666214383

Rule: spaghetti -> ground beef
Support: 0.006399146780429276
Confidence: 0.302443220500107

Rule: olive oil -> ground beef
Support: 0.004799360085321957
Confidence: 0.21818181818181817
Lift: 3.33316052582855

Rule: shrimp -> ground beef
Support: 0.005999200106652446
Confidence: 0.5232558139534884
Lift: 3.005315360233627

Rule: soup -> olive oil
Support: 0.005065991201173177
Confidence: 0.21965317919075145
Lift: 3.35563848698539

Rule: olive oil -> nan
Support: 0.006532462338354886
Confidence: 0.22171945701357465
Lift: 4.221110779337115

Rule: spaghetti -> frozen vegetables
Support: 0.004532728969470737
Confidence: 0.28813559322033894
Lift: 3.0228043143297376

Name: Alisha Khatri
Roll no: 03

LAB 2.6: Write a Python program using the PyFPGrowth to perform frequent pattern mining and association rule generation on a transaction dataset.

Theory:

FP-Growth is an efficient algorithm for frequent pattern mining without candidate generation. It builds a compact tree structure (FP-tree) and extracts frequent itemsets. It is faster than Apriori for large datasets. PyFPGrowth is used for implementation.

Code:

```
import pyfpgrowth

# Sample transactions
transactions = [
['Milk', 'Bread', 'Saffron'],
['Milk', 'Saffron'],
['Bread', 'Saffron', 'Wafer'],
['Bread', 'Wafer']
]

print("Transactions:")
print(transactions)

# Find frequent patterns
# support_threshold = 0.5 means item must appear in 50% of transactions
patterns = pyfpgrowth.find_frequent_patterns(
transactions,
0.5
)

print("\nFrequent Patterns:")
print(patterns)

# Generate association rules
# confidence_threshold = 0.8 means 80% confidence
rules = pyfpgrowth.generate_association_rules(
patterns,
0.8
)

print("\nAssociation Rules:")
print(rules)

print("\nName: Alisha Khatri")
print("Roll No: 03")
```

Output:

```
.. Transactions:
[['Milk', 'Bread', 'Saffron'], ['Milk', 'Saffron'], ['Bread', 'Saffron', 'Wafer'], ['Bread', 'Wafer']]

Frequent Patterns:
{('Milk',): 2, ('Milk', 'Saffron'): 2, ('Bread', 'Milk'): 1, ('Bread', 'Milk', 'Saffron'): 1, ('Wafer',): 2, ('Saffron', 'Wafer'): 1, ('Bread', 'Wafer')}

Association Rules:
{('Milk',): (('Saffron',), 1.0), ('Bread', 'Milk'): (('Saffron',), 1.0), ('Wafer',): (('Bread',), 1.0), ('Saffron', 'Wafer'): (('Bread',), 1.0)}

Name: Alisha Khatri
Roll No: 03
```

LAB 2.7: Write a Python program using the Machine Learning approach to predict diabetes using the Naive Bayes classifier.

Theory:

Naive Bayes is a probabilistic machine learning algorithm based on Bayes theorem. It assumes features are independent and is widely used for classification problems like medical diagnosis. Scikit-learn provides GaussianNB for continuous data classification.

Code:

```
import pandas as pd

from sklearn.naive_bayes import GaussianNB
from sklearn import metrics

# Load dataset
data = pd.read_csv('Diabetes.csv')
print("Dataset Preview")
print(data.head())

# Split dataset (70% train, 30% test)
split_index = int(len(data) * 0.7)
train_data = data[:split_index]
test_data = data[split_index:]

# Features and labels (Train)
X_train = train_data.drop('Diabetes', axis=1)
y_train = train_data['Diabetes']

# Features and labels (Test)
X_test = test_data.drop('Diabetes', axis=1)
y_test = test_data['Diabetes']

# Create Naive Bayes model
model = GaussianNB()

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Results
print("\nActual Values: ", list(y_test))
print("Predicted Values:", list(y_pred))

# Evaluation
print("\nConfusion Matrix:")
```

```

print(metrics.confusion_matrix(y_test, y_pred))
print("\nPerformance Metrics:")
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
print("Precision:", metrics.precision_score(y_test, y_pred))
print("Recall:", metrics.recall_score(y_test, y_pred))
print("F1 Score:", metrics.f1_score(y_test, y_pred))

```

Output:

```

.. Dataset Preview
   Pragnency  Glucose  Blod Pressure  Skin Thikness  Insulin  BMI  DFP  \
0           1        85             66             29        0  26.6  0.351
1           8       183             64              0        0  23.3  0.672
2           1        89             66             23       94  28.1  0.167
3           0       137             40             35      168  43.1  2.288
4           5       116             74              0        0  25.6  0.201

   Age  Diabetes
0    31         0
1    32         1
2    21         0
3    33         1
4    30         0

Actual Values:  [0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0]
Predicted Values: [np.int64(0), np.int64(0), np.int64(1), np.int64(1), np.int64(0), np.int64(1), np.int64(0), np.int64(0), np.int64(1), np.int64(1), np.

Confusion Matrix:
[[128  24]
 [ 30  49]]

Performance Metrics:
Accuracy: 0.7662337662337663
Precision: 0.6712328767123288
Recall: 0.620253164556962
F1 Score: 0.6447368421052632

```

LAB 2.8: Write a Python program using the Machine Learning technique of clustering to group data points using the K-Means algorithm.

Theory:

K-Means is an unsupervised learning algorithm that groups data into K clusters based on similarity. It minimizes distance between data points and cluster centroids. It is widely used for segmentation problems. Scikit-learn provides KMeans implementation.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
# Generate random dataset (100 points, 2 features)
data = 100 * np.random.rand(100, 2)
# Apply K-Means
model = KMeans(n_clusters=3, init='random', n_init=10, random_state=42)
model.fit(data)
# Get cluster centers and labels
centers = model.cluster_centers_
labels = model.labels_
print("Cluster Centers:\n", centers)
print("\nCluster Labels:\n", labels)
# Plot clusters
colors = ["r", "g", "b"]
markers = ["+", "x", "*"]
for i in range(len(data)):
    plt.plot(
        data[i][0],
        data[i][1],
        color=colors[labels[i]],
        marker=markers[labels[i]]
    )
# Plot cluster centers
plt.scatter(
    centers[:, 0],
    centers[:, 1],
    marker="o",
```

```

s=200,
linewidths=3,
color="black"
)
plt.title("K-Means Clustering Result")
plt.show()

```

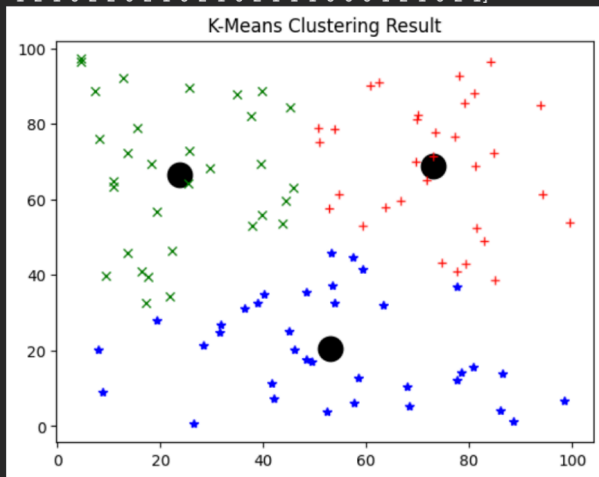
Output:

```

.. Cluster Centers:
  [[73.04737978 68.72669947]
   [23.72331487 66.51023909]
   [53.06786711 20.59711304]]

Cluster Labels:
[0 2 0 0 1 2 2 2 0 1 0 2 1 1 1 2 0 1 2 0 1 0 2 1 1 1 2 0 2 1 2 0 0 2 1 0 2
 2 1 2 0 2 2 1 1 0 0 2 0 1 0 0 0 1 1 0 2 2 0 0 2 2 2 1 1 0 2 2 0 2 2 1 2 1
 1 2 1 0 2 2 0 2 1 0 2 1 0 2 1 1 1 0 0 0 1 2 1 0 2 1]

```



LAB 2.9: Write a Python program using the Machine Learning technique of clustering to perform Mini-Batch K-Means clustering.

Theory:

Mini-Batch K-Means is a faster version of K-Means that uses small random batches of data instead of full dataset. It reduces computation time while maintaining good accuracy. It is useful for large datasets.

Code:

```
import time
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import MiniBatchKMeans
# Generate random dataset (100 points, 2 features)
data = 100 * np.random.rand(100, 2)
print("Sample Data:\n", data[:10])
# Create Mini-Batch K-Means model
model = MiniBatchKMeans(
    n_clusters=5,
    init='random',
    batch_size=20,
    n_init=1,
    random_state=42
)
# Measure training time
start_time = time.time()
model.fit(data)
end_time = time.time()
print("\nTotal Time Taken:", end_time - start_time, "seconds")
# Get centers and labels
centers = model.cluster_centers_
labels = model.labels_
print("\nCluster Centers:\n", centers)
print("\nLabels:\n", labels)
# Plot clusters
colors = ["g", "r", "b", "y", "m"]
markers = ["+", "x", "*", ".", "d"]
for i in range(len(data)):
    plt.plot(
```

```

data[i][0],
data[i][1],
color=colors[labels[i]],
marker=markers[labels[i]]
)
# Plot centroids
plt.scatter(
centers[:, 0],
centers[:, 1],
marker="o",
s=200,
linewidths=3,
color="black"
)
plt.title("Mini-Batch K-Means Clustering")
plt.show()

```

Output:

```

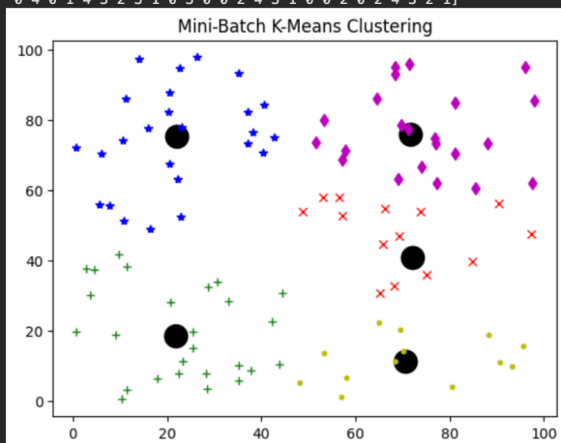
... Sample Data:
[[71.30935515 77.34706315]
[57.88759205 71.36618477]
[64.59879123 86.10231103]
[42.262269 22.68459308]
[20.29634673 82.37180469]
[95.71299513 15.64842483]
[85.47743347 60.53606313]
[75.13399271 35.90814625]
[10.33831586 0.3520333 ]
[68.4618009 95.10455568]]

Total Time Taken: 0.015284538269042969 seconds

Cluster Centers:
[[21.86098585 18.52873718]
[72.06090895 40.88751621]
[22.0952539 75.47144887]
[70.60598007 11.27369152]
[71.74058841 75.94160996]]

Labels:
[4 4 4 0 2 3 4 1 0 4 0 3 4 4 0 4 1 2 2 2 0 4 2 2 2 3 0 2 3 0 1 2 1 2 1 1
1 4 2 2 3 4 0 4 4 2 0 2 3 4 2 2 1 0 1 2 0 4 0 3 2 2 0 0 4 4 1 2 0 0 3 0 0
0 4 0 1 4 3 2 3 1 0 3 0 0 2 4 3 1 0 0 2 0 2 4 3 2 1]

```



LAB 2.10: Write a Python program using the Machine Learning technique of Hierarchical Clustering (Agglomerative Clustering) to group data points.

Theory:

Hierarchical clustering builds a tree-like structure of clusters called a dendrogram. Agglomerative clustering starts with individual points and merges them step by step. It helps visualize cluster relationships. SciPy is used for dendrogram generation.

Code:

```
import matplotlib.pyplot as plt
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster import hierarchy
# Sample data points
x = [4, 5, 10, 4, 3, 11, 14, 6, 10, 12]
y = [21, 19, 24, 17, 16, 25, 24, 22, 21, 21]
# Combine x and y into dataset
data = list(zip(x, y))
# Apply Agglomerative Clustering
model = AgglomerativeClustering(
    n_clusters=2,
    metric='euclidean',
    linkage='single'
)
labels = model.fit_predict(data)
print("Cluster Labels:", labels)
# Create linkage matrix for dendrogram
linkage_matrix = hierarchy.linkage(data, method='ward')
# Plot dendrogram
plt.figure(figsize=(8, 5))
hierarchy.dendrogram(linkage_matrix)
plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()
# Optional: Scatter plot of clusters
plt.scatter(x, y, c=labels, cmap='rainbow')
plt.title("Agglomerative Clustering Result")
```

```
plt.show()
print("\nName: Alisha Khatri")
print("Roll No: 03")
```

Output:

